PROJETO DE PESQUISA — TRILHAS DE PTT

Arquitetura e Organização de Computadores · Mestrado Profissional em Computação Aplicada (PPCA/UFRR)

Projeto de PTT4 preenchido para avaliação e aprovação antes do início da execução.

PARTE I — MODELO A SER PREENCHIDO

Este projeto formaliza a proposta de uma biblioteca embarcada para ESP32, com foco na integração de dados locais e meteorológicos externos.

Discente(s): Ezequias Carlos Ferreira da Silva Marcelo Gomes Barbosa	Dupla? (S/N): S
Orientador(a): Herbert Oliveira Rocha	Semestre: 2026.2
Prazo: 11 semanas	Data: 26/11/2026
Linha de pesquisa: ☑ Sistemas Embarcados/Distribuídos	☐ Informática na Educação

1. TÍTULO DO PROJETO (PROVISÓRIO)

Biblioteca Rust para Sistemas IoT Embarcados Context-Aware: Integração de Dados Ambientais e Tolerância a Falhas

2. INTRODUÇÃO E JUSTIFICATIVA

2.1 Contextualização

Sistemas IoT embarcados utilizam sensores para coletar dados do ambiente, mas a integração de diferentes fontes pode introduzir desafios relacionados à heterogeneidade, qualidade, consistência e veracidade dos dados (DING et al., 2019; CHHETRI et al., 2024). A utilização de informações externas como contexto pode complementar as leituras locais e apoiar a interpretação dos dados, especialmente quando diferentes variáveis apresentam relações temporais ou espaciais (DUQUE et al., 2025).

Entretanto, a combinação de fontes não garante, por si só, maior qualidade dos dados. Sensores podem apresentar leituras inválidas ou ausentes, enquanto fontes externas podem sofrer indisponibilidade ou apresentar informações divergentes. Estudos sobre qualidade de dados em IoT destacam, entre outras dimensões, precisão, completude, consistência e atualidade como aspectos relevantes para a confiabilidade das informações (CHHETRI et al., 2024).

A disponibilidade das fontes também influencia a confiabilidade de sistemas IoT. Em sistemas embarcados conectados, falhas de leitura do sensor, indisponibilidade da rede ou ausência temporária de uma fonte externa exigem comportamento explicitamente definido, evitando a propagação silenciosa de dados inválidos e permitindo a continuidade controlada da operação quando possível.

Nesse contexto, Rust apresenta características relevantes para software embarcado, especialmente por oferecer segurança de memória, abstrações de baixo nível e suporte a desenvolvimento com recursos restritos. O ecossistema embedded Rust, incluindo mecanismos de abstração de hardware, permite estruturar bibliotecas reutilizáveis e com interfaces explícitas para periféricos, sem afastar o desenvolvedor das restrições de memória, processamento e entrada/saída da plataforma.

Assim, este trabalho investiga como uma biblioteca embarcada em Rust pode integrar dados ambientais locais e externos de forma controlada, mantendo comportamento definido diante de falhas parciais e permitindo avaliar, no ESP32, corretude dos registros, latência das operações e uso de recursos de memória.

2.2 Problema de pesquisa

Como projetar e avaliar uma biblioteca embarcada em Rust para ESP32 capaz de integrar dados ambientais locais e externos, manter comportamento definido diante da indisponibilidade parcial das fontes e apresentar custos mensuráveis de latência e memória?

2.3 Hipótese(s) ou pergunta(s) norteadora(s) (opcional)

Q1. Quais mecanismos de validação e integração são necessários para assegurar a corretude de dados ambientais provenientes de fontes locais e externas em uma biblioteca embarcada?

Q2. Como a biblioteca deve tratar a indisponibilidade de uma das fontes para manter uma operação definida, incluindo modo degradado e recuperação?

Q3. Quais evidências experimentais permitem avaliar a corretude dos registros, a detecção e recuperação de falhas, a latência das operações e o uso de RAM e Flash da biblioteca no ESP32?

3. OBJETIVOS

3.1 Objetivo geral

Desenvolver e validar uma biblioteca embarcada em Rust para ESP32 destinada à integração de dados ambientais locais e externos, com mecanismos explícitos de validação, operação degradada e recuperação, avaliando experimentalmente corretude dos registros, latência e uso de recursos de memória.

3.2 Objetivos específicos (3 a 5 itens)

1. Especificar os requisitos funcionais, o modelo de dados e as propriedades esperadas de validade, consistência e tratamento de falhas na integração de fontes ambientais locais e externas.
2. Projetar uma arquitetura modular para a biblioteca, utilizando abstrações de hardware compatíveis com o ecossistema embedded Rust e separando aquisição, validação, integração e estados de operação.
3. **Implementar** a biblioteca em Rust e integrá-la a uma plataforma embarcada, utilizando o ESP32, o sensor DHT11 como fonte local e a API Open-Meteo como fonte externa.
4. Verificar o comportamento da biblioteca por meio de testes automatizados e análise estática, cobrindo propriedades relacionadas à validade dos dados e às transições entre estados normal, degradado, erro e recuperação.
5. Validar experimentalmente a biblioteca em cenários de operação normal, falhas parciais e recuperação, avaliando corretude dos registros, detecção de falhas, latência e utilização de RAM e Flash.

4. TRILHA DE PTT ESCOLHIDA

☐ PTT1 — Protótipo de Sistema Ciberfísico (CPS)	☐ PTT2 — IP Core em VHDL
☐ PTT3 — Framework de aceleração paralela (CUDA)	☑ PTT4 — Biblioteca de software embarcado
☐ PTT5 — PoC de coprocessador / extensão de ISA	☐ PTT6 — Benchmarking e eficiência energética
☐ PTT7 — Controle baseado em FSM	☐ PTT8 — Depósito de patente/registro (INPI)*
☐ PTT9 — Simulador de microarquitetura c/ telemetria	☐ PTT10 — Bancada de lab. remoto FPGA/GPU/MCU
☐ PTT11 — Avaliador automático de assembly	

** PTT8 apenas como produto secundário, nunca como principal.*

4.1 Justificativa da escolha da trilha

PTT4 — Biblioteca de software embarcado — A trilha permite concentrar o trabalho na especificação, implementação, verificação e validação de um artefato de software embarcado com escopo definido e potencial de reutilização. A escolha possibilita explorar programação embarcada, abstração de hardware, integração de fontes de dados, tratamento de falhas e mensuração de custos computacionais, tendo o ESP32 como plataforma experimental. O DHT11 e a Open-Meteo constituem as fontes do estudo de caso, enquanto a contribuição se concentra na biblioteca e no comportamento que ela apresenta diante da disponibilidade ou indisponibilidade dessas fontes. A API documentada, a suíte de testes e os experimentos de latência e memória permitirão relacionar o software desenvolvido às restrições concretas da arquitetura embarcada.

5. FUNDAMENTAÇÃO E VÍNCULO COM A EMENTA

O projeto fundamenta-se em conceitos de sistemas embarcados, arquitetura e organização de computadores e desenvolvimento de software de baixo nível, considerando a interação entre software, processador, memória e dispositivos de entrada e saída. A biblioteca será desenvolvida em Rust para execução no ESP32, exigindo o tratamento de restrições de memória e processamento, comunicação com periféricos e abstrações de acesso ao hardware. A avaliação de latência, RAM e Flash conecta diretamente o comportamento do artefato aos recursos computacionais disponíveis na plataforma.

O vínculo com a ementa ocorre principalmente por meio dos conceitos de **E/S e interrupções**, utilizados na comunicação com o sensor e nos mecanismos de temporização; **mapeamento e abstração de recursos de hardware**, necessários à interação entre a biblioteca e a plataforma; e **arquitetura e organização do processador**, incluindo aspectos relacionados à ISA e à aritmética utilizada no processamento e validação dos dados. A biblioteca também empregará **máquinas de estados** para representar estados de operação, falha e recuperação das fontes de dados. Esses elementos serão integrados ao desenvolvimento e à verificação do artefato, permitindo relacionar os conceitos estudados na disciplina com sua aplicação prática em software embarcado.

6. METODOLOGIA

6.1 Escopo mínimo viável

A implementação de referência será realizada em um ESP32, utilizando o DHT11 como fonte local de temperatura e umidade e a Open-Meteo como fonte externa de dados meteorológicos. A solução deverá identificar falhas de leitura ou comunicação, registrar a origem e o estado de validade dos dados, evitar a propagação silenciosa de valores inválidos e manter comportamento definido quando uma das fontes estiver indisponível. O artefato será acompanhado de documentação da API, exemplos de utilização, suíte de testes automatizados, dados experimentais estruturados e relatório técnico.

6.2 Escopo excluído explicitamente

Não fazem parte do escopo deste trabalho a implementação do protocolo de comunicação de baixo nível do DHT11, a criação de hardware dedicado ou de uma nova arquitetura de processamento, o desenvolvimento de uma solução completa de automação ou monitoramento ambiental, nem a medição de consumo energético. Serão utilizadas bibliotecas e abstrações existentes para acesso aos periféricos e recursos da plataforma, concentrando a contribuição na biblioteca, na integração das fontes, no tratamento de falhas e na avaliação de latência e memória.

6.3 Protocolo experimental

A avaliação será conduzida por meio de testes automatizados e experimentos controlados no ESP32, considerando diferentes condições de disponibilidade das fontes. Inicialmente serão definidos os requisitos e propriedades que caracterizam o comportamento esperado da biblioteca: validade dos dados, identificação de origem, tratamento de valores inválidos e transições entre os estados normal, degradado, erro e recuperação. Essas propriedades serão utilizadas como critérios de teste. Em

cada cenário também serão coletadas métricas de latência das operações e de utilização de recursos de memória da aplicação.

Os experimentos serão organizados, no mínimo, nos seguintes cenários:

Cenário	DHT11	Open-Meteo	Comportamento esperado
C1 — Operação normal	Disponível	Disponível	Integração das fontes e produção de registro válido
C2 — Falha local	Indisponível	Disponível	Identificação da falha e operação degradada
C3 — Falha externa	Disponível	Indisponível	Identificação da falha e operação degradada
C4 — Falha simultânea	Indisponível	Indisponível	Estado de erro explicitamente identificado
C5 — Recuperação	Recuperado	Recuperado	Retorno ao estado normal sem inconsistências

Métricas principais de avaliação

Métrica	Forma de observação	Finalidade
Latência	Tempo das operações principais da biblioteca em cada cenário	Avaliar custo temporal e variação entre condições de operação
Confiabilidade funcional	Sucesso/erro, estado produzido e recuperação	Verificar se o comportamento observado corresponde ao especificado
RAM	Memória utilizada durante a execução	Quantificar o custo de memória volátil da solução
Flash	Tamanho final do firmware/aplicação	Quantificar o custo de armazenamento do artefato embarcado

Para cada cenário serão coletados os resultados produzidos pela biblioteca, as ocorrências de erro e as transições de estado. Serão avaliados, conforme aplicável, validade e consistência dos registros, detecção de falhas, operação em modo degradado, recuperação, latência das operações e uso de RAM e Flash. A Open-Meteo será utilizada como fonte externa de contexto e não como padrão absoluto de verdade para as leituras do DHT11. Os resultados serão armazenados em formato estruturado, preferencialmente CSV, para posterior análise.

7. INFRAESTRUTURA NECESSÁRIA

☐ Kit Raspberry Pi Zero W

☑ ESP32 / Arduino Mega

☐ FPGA + Quartus Prime Lite

☐ Estação com GPU RTX 3090

☑ Sensores (especificar abaixo)

Outro / especificação: ESP32 DevKit; módulo DHT11; protoboard; jumpers; cabo USB; rede Wi-Fi; computador com VS Code, toolchain Rust/ESP, Git e acesso à Open-Meteo.

8. CRONOGRAMA (11 SEMANAS)

Fase	Semanas	Atividades principais
Revisão e protocolo	1–2	Revisão técnica; definição da API, requisitos, estados, cenários e métricas.
Implementação	3–5	Leitura do DHT11; integração Open-Meteo; modelo de dados; estados normal, degradado, erro e recuperação.
Testes e instrumentação	6–7	Testes automatizados; instrumentação de latência; levantamento de RAM e Flash; validação dos cenários.
Execução dos experimentos	8–9	Execução repetida dos cenários normal, falhas parciais, falha simultânea e recuperação; coleta dos dados.
Análise e redação	10	Análise das métricas; documentação da API; README e relatório técnico.
Vídeo e entrega final	11	Demonstração funcional, revisão do repositório e entrega final em 26/11/2026.

9. RISCOS E PLANO DE CONTINGÊNCIA

Use como referência o Plano B sugerido no catálogo para a trilha escolhida.

Risco principal	Probabilidade	Impacto	Plano B
Dificuldades na integração do Rust com o ESP32	Média	Alto	Restringir inicialmente a implementação às abstrações e funcionalidades essenciais, utilizando HALs e bibliotecas existentes e bem estabelecidas.
Indisponibilidade ou instabilidade da Open-Meteo/rede Wi-Fi durante os testes	Média	Médio	Utilizar respostas simuladas ou gravadas para os testes automatizados e manter a API externa apenas como fonte de validação experimental.
Tempo insuficiente para implementação, testes e validação	Média	Alto	Priorizar o núcleo funcional, os testes e os cenários de falha definidos no escopo mínimo, postergando funcionalidades complementares.

10. RESULTADOS ESPERADOS E ENTREGÁVEIS

10.1 Entregáveis-base (obrigatórios para qualquer trilha)

- Repositório versionado com licença
- README de reprodutibilidade
- Dados brutos
- Vídeo de demonstração (3–5 min)
- Relatório técnico conclusivo

10.2 Consolidação pós-disciplina

Publicação do código em repositório público e, se o nível de maturidade permitir, disponibilização da biblioteca como crate; possibilidade posterior de registro de software e publicação do conjunto de dados experimentais.

11. REFERÊNCIAS BIBLIOGRÁFICAS PRELIMINARES

AOSONG (GUANGZHOU) ELECTRONICS CO., LTD. **DHT11 Product Manual: Temperature and Humidity Module**. [S.l.]: Aosong, [s.d.]. Disponível em: https://projectpoint.in/datasheets/pdf/DHT11.pdf. Acesso em: 9 set. 2026.

CHHETRI, Tek Raj et al. **Enabling privacy-aware interoperable and quality IoT data sharing with context**. *Future Generation Computer Systems*, v. 157, p. 164–179, 2024. DOI: 10.1016/j.future.2024.03.039.

DING, Wenxiu et al. **A survey on data fusion in internet of things: Towards secure and privacy-preserving fusion**. *Information Fusion*, v. 51, p. 129–144, 2019. DOI: 10.1016/j.inffus.2018.12.001.

DUQUE, Hector et al. **Context-aware data driven sensor data analysis: With application to H2S concentration prediction in urban drainage networks**. *Water Research X*, v. 28, 100346, 2025. DOI: 10.1016/j.wroa.2025.100346.

ESPRESSIF SYSTEMS. **ESP32 Technical Reference Manual**. Disponível em: https://www.espressif.com/en/document-type/reference. Acesso em: 9 set. 2026. Documento de referência para os recursos de memória e periféricos utilizados na plataforma ESP32.

RUST FOUNDATION. The Embedded Rust Book. Disponível em: https://doc.rust-lang.org/stable/embedded-book/. Acesso em: 9 set. 2026. A obra aborda o desenvolvimento de software embarcado em Rust, incluindo ambientes bare metal, no_std, acesso a periféricos, interrupções e abstração de hardware.

OPEN-METEO. **Open-Meteo API Documentation**. Disponível em: https://open-meteo.com/en/docs. Acesso em: 9 set. 2026.

RUST-EMBEDDED. **embedded-hal: A Hardware Abstraction Layer for embedded systems**. Disponível em: https://docs.rs/embedded-hal/latest/embedded_hal/. Acesso em: 9 set. 2026. A biblioteca fornece traits para abstração de periféricos e constitui uma base para o desenvolvimento de drivers e bibliotecas independentes de plataforma.

STALLINGS, William. Arquitetura e organização de computadores. 10. ed. São Paulo: Pearson Education do Brasil, 2017.

gov.br Documento assinado digitalmente
EZEQUIAS CARLOS FERREIRA DA SILVA
Data: 15/09/2026 19:32:33-0300
Verifique em https://validar.iti.gov.br

Assinatura do(s) discente(s)

Assinatura do(a) orientador(a)

gov.br Documento assinado digitalmente
MARCELO GOMES BARBOSA
Data: 15/09/2026 19:37:31-0300
Verifique em https://validar.iti.gov.br